

User Story Revision Draft

Proposed changes to the backlog, derived from the eleven open defects and the requirement gaps found during QA. **Nothing here has been written to Azure DevOps** — this is for your review first.

PREPARED BY Shanthi Subramanian · BA / QA	DATE 17 September 2026	STATUS Draft — awaiting confirmation	DAYS TO SUBMIT 13
---	--	--	---

9 NEW CRITERIA	7 NEW STORIES	0 STORIES DELETED	3 BLOCKED ON DECISIONS
--------------------------------------	-------------------------------------	---	--

Before you read this

Two things you need to know about how this draft was produced.

I CANNOT SEE YOUR AZURE DEVOPS BOARD

There is no Azure DevOps connector available to me in this session, and the Azure CLI is not installed on this machine. **This draft was written against `docs/requirements/user-stories.md` in the repository**, which I have read in full. If the board has diverged from that document, the draft needs reconciling against the board before anything is applied — and you are the one who can see both.

What I changed, and why only this much

Every amendment below traces to something concrete — an open defect, or a gap the existing document names as its own next step. I have **not** rewritten stories that are working, and I have **not** invented requirements to look thorough. Nine new acceptance criteria and seven new stories, each with a stated origin.

The existing document is good. Its Given/When/Then criteria are directly testable, the assumptions are called out inline, and the traceability matrix at the end is more than most teams produce. The gaps are the ones it already admits to, plus the ones our testing exposed.

Reading the markup

MARKER	MEANING
AMEND	Existing story, new acceptance criteria appended. Nothing existing is removed.
NEW	A story that does not exist today.
+ highlighted line	The specific new criterion to add.

Part A — new criteria on existing stories

Five stories, nine new acceptance criteria. Each closes a defect our tests found.

US-6.3 · Epic 6

Finalize the assessment

AMEND

DEF-004

HIGH

As an FCRM Analyst, I want to formally mark an assessment as finalized, so that it can move to committee only once I've reviewed everything.

- ▶ Given an assessment has outstanding AI-drafted sections not yet reviewed, when I try to finalize, then the system blocks finalization and lists what's outstanding. EXISTING
- ▶ Given all sections are reviewed, when I finalize, then the assessment locks and enters "Pending Committee". EXISTING
- ▶ Given I finalize, when this occurs, then my identity and timestamp are recorded — never anonymous or "system". EXISTING

+ **Given an assessment has no risk categories mapped at all**, when I try to finalize, then the system blocks it and tells me at least one category must be mapped — readiness must never be satisfied vacuously.

+ **Given every proposed category has been deactivated by me**, when I try to finalize, then the system blocks it for the same reason.

Origin: DEF-004. Today, an assessment with zero categories, zero policy review, zero narrative and zero scores reports "Ready to finalize", finalizes, routes, and appears in the committee queue with a live vote button. Confirmed end to end.

US-8.2 · Epic 8

Cast a committee vote

AMEND DEF-002 HIGH

As a Risk Committee Member, I want to review a finalized assessment and cast a vote, so that a formal, accountable decision is recorded.

- ▶ Given I open an assessment in my queue, then I can see narrative, scores, cited policy and override history before voting. EXISTING
- ▶ Given "approve-with-conditions", then conditions are mandatory structured text. EXISTING
- ▶ Given multiple members vote, then each vote and rationale is recorded separately. EXISTING

+ **Given I do not hold the Committee Member role**, when a vote is submitted in my name, then the system rejects it with an authorization error — the acting user is established from the authenticated session, never accepted from the request payload.

Origin: DEF-002. Every write endpoint currently reads the acting user from the request body and performs no role check, so any caller can act as any user.

US-7.1 · Epic 7

View calculated risk score with breakdown

AMEND DEF-005 MEDIUM

As an FCRM Analyst, I want to see the calculated risk score along with how it was derived, so that the number is explainable, not a black box.

- ▶ Given mapped categories, then I see inherent and residual ratings per category. EXISTING
- ▶ Given any control applied, then residual is always greater than zero. EXISTING
- ▶ Given a residual score, then I can trace which controls were credited and each one's weight. EXISTING

+ **Given I am scoring a category**, when I open the controls picker, then I see a non-empty catalogue of named controls for that category, each with a description — so that "which controls were credited" is answerable rather than always empty.

Origin: DEF-005. The `control` table is created but never seeded, so the controls endpoint returns an empty list for every category and the third existing criterion above cannot currently be met.

US-2.1 · Epic 2

Auto-map request to risk categories

AMEND DEF-006 MEDIUM

As an FCRM Analyst, I want the system to propose which supervisory risk categories apply, so that I don't start from a blank page.

- ▶ Given a submitted request, then the system proposes categories from a named framework with citations. EXISTING
- ▶ Given no framework mapping is configured, then the system flags this rather than guessing silently. EXISTING

+ **Given I open the assessment workspace for a request**, when the workspace loads — including when two analysts open it simultaneously — then it opens successfully first time, without an error, and exactly one assessment exists for that request.

Origin: DEF-006. `func_getOrCreateAssessment` does a select-then-insert with no conflict handling, so concurrent callers race and one receives a 500. In practice the first click fails every time.

US-1.3 · Epic 1

Track my request status

AMEND DEF-003 MEDIUM

As a Product Owner, I want to see where my request stands, so that I don't have to email FCRM for updates.

- ▶ Given I have submitted a request, then I see status and days elapsed. EXISTING
- ▶ Given a status change, then I receive a notification. EXISTING

+ **Given I request something that does not exist or no longer exists**, when the system responds, then I get a clear "not found" message — the system never reports my own bad input as an internal server error.

+ **Given any error occurs**, when it is shown to me, then it explains what went wrong and what to do next, and never exposes a database or stack-trace detail.

Origin: DEF-003. A well-formed but unknown id reaches Postgres and the foreign-key violation escapes as HTTP 500, so "you sent an id that doesn't exist" reads to the caller as "the server broke".

Part B — new stories

Seven stories covering ground the backlog does not currently reach. Two are epics of their own.

US-11.1 · new Epic 11 – Access Control

Enforce role-based permissions on every action

NEW

DEF-001, DEF-002

HIGH

As FCRM Leadership, I want every action to be permitted only to the role that owns it, so that the accountability the audit trail records is real rather than assumed.

- + Given any request to a protected endpoint, when no valid credential is presented, then the request is rejected — no data of any kind is returned.
- + Given an authenticated user, when they attempt an action their role does not permit, then it is refused on authorization grounds, before any record is read or written.
- + Given any write action, when the acting user is determined, then it comes from the authenticated session — never from a field in the request payload.
- + Given the permission model, when it is reviewed, then a documented matrix maps each of the three roles to every action, and each endpoint's behaviour matches it.

Origin: DEF-001 and DEF-002. Today the user directory is readable with no credential at all, and the UI hides screens by role while the API enforces nothing — which is worse than having no role model, because the product looks access-controlled.

Export an examiner-ready audit trail

NEW HIGH

As an Auditor or FCRM Analyst preparing for examination, I want to export a request's complete history in a durable format, so that I can hand it to an examiner without reformatting it first.

- + Given any request id, when I export its history, then the export contains every event in order — submission, uploads, AI outputs, human overrides with reasons, policy citations relied upon, score calculations, routing, individual votes and the final decision.
- + Given an export, when I open it, then each entry shows actor, exact timestamp, and the before and after state.
- + Given an export, when it is produced, then it is a durable human-readable document requiring no further formatting.
- + Given an assessment with AI-generated content, when exported, then both the original AI output and any human override appear, never the override alone.

Origin: US-9.1 AC3 states this requirement but no story owns the export itself, so nothing implements or tests it. This is the deliverable an examiner actually receives.

US-12.1 · new Epic 12 – Deployment & Operations

Apply the database schema to a deployed environment

NEW DEF-007 HIGHEST

As the DevOps engineer, I want the schema, stored functions and reference data applied automatically to a target environment, so that a deployed application can actually serve a request.

- + Given a newly provisioned database, when the release runs, then schema, all `func_*` routines and reference seed data are applied before the application containers start accepting traffic.
- + Given the schema deployment runs twice, when it completes, then the second run is a safe no-op and destroys no data.
- + Given schema deployment fails, when it does, then the release halts and reports which statement failed — it never proceeds to start an application against an incomplete schema.
- + Given a deployed environment, when I verify it, then a documented post-deploy check confirms the expected function and seed-row counts.

Origin: DEF-007. The DB folder is not copied into either image, the application performs no bootstrap, and no pipeline applies it. On first deploy the containers will start cleanly and fail on every request.

US-12.2 · Epic 12

Build and publish container images automatically

NEW DEF-009, DEF-011 MEDIUM

As the DevOps engineer, I want images built, scanned and published by the pipeline, so that what reaches Azure is reproducible and has been checked.

- + Given a merge to the main branch, when the pipeline runs, then both service images build from the repository root and are pushed to the registry, tagged with the commit.
- + Given an image is built, when it is scanned, then no HIGH or CRITICAL operating-system vulnerability is present, and the build fails if one is.
- + Given a running container, when its user is inspected, then it is not root.
- + Given any image, when its layers are inspected, then no local configuration file or secret is present.

Origin: DEF-009 and DEF-011. Both Dockerfiles currently omit a `USER` directive so every process runs privileged; there is no image scanning because there is no application pipeline at all.

US-12.3 · Epic 12

Observe a running environment

NEW DEF-010 MEDIUM

As the DevOps engineer, I want logs, traces and metrics from a deployed environment, so that a failure can be diagnosed without redeploying to reproduce it.

- + Given a deployed environment, when a request is served, then its trace is queryable within minutes, including outbound calls to the database, Mock Systems and the AI provider.
- + Given an unhandled error, when it occurs, then it is recorded with enough context to identify the request, without recording assessment content or personal data.
- + Given cost constraints, when telemetry is configured, then a sampling rate is set deliberately and documented.

Origin: DEF-010. Both services already register OpenTelemetry and activate on a connection string, but the Log Analytics workspace is commented out in Terraform – so a deployed environment would emit nothing.

US-13.1 · new Epic 13 – Non-Functional

Retain assessment records for the supervisory period

NEW MEDIUM

As FCRM Leadership, I want assessment records and their audit trail retained for the full supervisory retention period, so that an examiner can review a decision years after it was taken.

- + Given a finalized assessment, when the retention period is defined, then it is at least five years and is stated in documentation rather than assumed.
- + Given the database, when backup is configured, then point-in-time restore is enabled and the retention window is documented.
- + Given an attached document, when the request is decisioned, then the document remains retrievable for the same period as its assessment.

Origin: the existing document names non-functional requirements as its own next step, and retention is the one with a regulatory floor. Currently undefined – no backup or retention policy exists on the database.

Keep model calls inside the tenant

NEW

MEDIUM

As FCRM Leadership, I want AI calls to stay within our own Azure tenant, so that assessment content never leaves our control boundary.

- + Given a deployed environment, when an AI call is made, then it goes to the in-tenant Azure AI Foundry deployment, not a third-party endpoint.
- + Given configuration, when the environment starts, then a non-production provider cannot be selected by default in a deployed environment.
- + Given any AI call, when it is logged, then the prompt and response content are not written to general application logs.

Origin: data residency was previously abstract; with Foundry now provisioned it is decidable. The provider is currently selected by a configuration value with a non-Azure default.

Part C — blocked on a team decision

Three criteria I have deliberately not written, because writing them would be inventing policy.

Each of these changes the design depending on the answer. I would rather bring you three open questions than three plausible-sounding criteria that turn out to be wrong.

QUESTION	AFFECTS	IF ANSWERED "YES"
Q3 — does a scoring override need secondary sign-off before finalization?	US-7.2, US-6.3	Adds a review state and a second actor to the finalize gate — a schema change, not just a criterion
Q5 — who may approve scoring and workflow configuration changes?	US-10.1, US-10.2	Adds an approval workflow to configuration, and a fourth permission tier to the RBAC matrix
Q6 — who owns database schema deployment? NEW	US-12.1	Determines whether the schema ships in the release pipeline or as an application concern

Q6 IS THE URGENT ONE

Q3 and Q5 can reasonably be deferred and documented as accepted scope decisions. Q6 cannot — it is an unassigned responsibility sitting between two owners, which is exactly why it fell through the gap. It needs a name against it before the first Azure deploy, not during it.

How to load this into Azure DevOps

Three options, depending on how much you want automated.

Option 1 — CSV import FASTEST

A ready-to-import file accompanies this draft at `docs/qa/ado-user-stories-draft.csv`. In Azure DevOps: **Boards** → **Work Items** → **Import Work Items**, then map `Work Item Type`, `Title`, `Description`, `Acceptance Criteria`, `Tags` and `Priority`.

Leave the `ID` column empty for new items so ADO creates them. To *amend* an existing story rather than create a duplicate, put its real ADO id in that column first — otherwise Part A will land as seven new work items instead of updating five.

Option 2 — give me CLI access BEST FOR ONGOING WORK

Install the Azure CLI and its DevOps extension, then sign in yourself:

```
winget install Microsoft.AzureCLI
az extension add --name azure-devops
az login
az devops configure --defaults organization=https://dev.azure.com/<org> project=<project>
```

After that I can read the real board and apply changes directly with `az boards work-item update`, which removes the guesswork about what is already there. You run `az login` — I never handle your credentials.

Option 3 — paste the board here

Export the current stories and paste them in. I will reconcile this draft against what actually exists and produce a precise change list. Slowest, but it needs no tooling.

MY RECOMMENDATION

Option 2, because this will not be the last backlog update before submission and the board is the single source of truth. Option 1 as an immediate fallback if installing tooling is a nuisance today.

Draft only — nothing has been written to Azure DevOps. Prepared against `docs/requirements/user-stories.md` at commit `a816378`; if the board has diverged from that document, reconcile before applying. Every amendment traces to a defect verified against a running local stack, or to a gap the existing requirements document names itself.